INTRO TO ALGO ENGINEERING

Project 1

Biconnected Components

Comparing

Jens-Schmidt Chain Decomposition

vs

Slota–Madduri Biconnectivity

Team Members

Manik Bansal 2024101084

Vansh Goyal 2024101044

Contents

1 Problem Statement **2**

2 Algorithm Descriptions **2**
2.1 Jens-Schmidt Chain Decomposition . . . 2
2.1.1 Key Idea . . . 2
2.1.2 How It Works — Example . . . 2
2.1.3 Complexity . . . 3
2.2 Slota–Madduri Biconnectivity . . . 3
2.2.1 Key Idea . . . 3
2.2.2 How It Works — Example . . . 3
2.2.3 Parallelisation . . . 4
2.2.4 Complexity . . . 4

3 Implementation Details **4**
3.1 Jens-Schmidt (`Jen-Schmidt-dataset.cpp`) . . . 4
3.2 Slota–Madduri (`slota-dataset.cpp`) . . . 5

4 Experimental Setup **5**
4.1 Dataset . . . 5
4.2 Methodology . . . 6

5 Results **6**
5.1 Dense Graphs ($m \approx n^2/2$, $V = 100$–1000) . . . 6
5.2 Highly Connected Graphs ($V = 100$–1000) . . . 6
5.3 Large Graphs ($V = 1000$–10000) . . . 6
5.4 Real-World Graphs . . . 7
5.5 Small Graphs ($V = 50$–500) . . . 7
5.6 Sparse Graphs ($m \approx 1.5n$, $V = 1000$–10000) . . . 8
5.7 Tree-Like Graphs ($m \approx n$, $V = 1000$–10000) . . . 8

6 Analysis: Expected vs. Observed Behaviour **8**
6.1 Time . . . 8
6.2 Memory . . . 8

7 Plots **9**

8 Conclusion **9**

1 Problem Statement

Given an undirected graph $G = (V, E)$, two fundamental questions arise:

1. **Is G biconnected?** A graph is *biconnected* if removing any single vertex leaves the graph connected.
2. **What are the articulation points?** An articulation point (cut vertex) is a vertex whose removal disconnects the graph.

These problems are central to network reliability analysis, circuit design, and planarity testing. We implement and compare two algorithms that solve these problems using fundamentally different strategies: **Jens-Schmidt** via chain decomposition (DFS-based, $O(V + E)$) and **Slota–Madduri** via exclusion-BFS (BFS-based, parallelizable).

2 Algorithm Descriptions

2.1 Jens-Schmidt Chain Decomposition

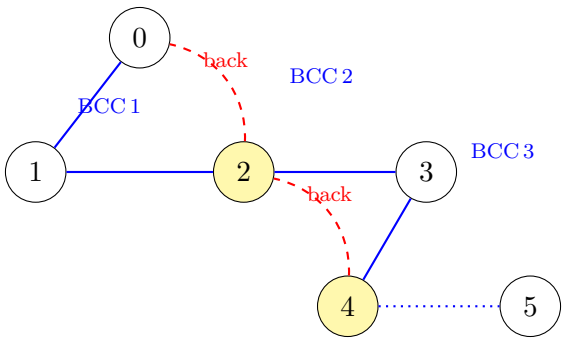
2.1.1 Key Idea

The algorithm builds a **chain decomposition** of an undirected graph G using a DFS tree. A *chain* is a path that starts with a back edge (u, v) (where v is a DFS ancestor of u) and continues upward along the DFS tree through unclaimed tree edges.

Theorem (Schmidt, 2013): G is biconnected if and only if every tree edge belongs to some chain. An unclaimed tree edge is either a bridge or borders an articulation point.

2.1.2 How It Works — Example

Consider the following 6-vertex graph with **3 biconnected components**: two triangles joined at an articulation point, plus a bridge.



Graph: vertices $\{0, 1, 2, 3, 4, 5\}$, edges 0−1, 0−2, 1−2, 2−3, 2−4, 3−4, 4−5. Yellow vertices are articulation points; dotted blue edge is a bridge.
Step 1: DFS from vertex 0 (visiting neighbours in order).

- Tree edges: $0 \to 1 \to 2 \to 3 \to 4 \to 5$
- Back edges discovered: $(2, 0)$ and $(4, 2)$
- Depths: $d[0]$=0, $d[1]$=1, $d[2]$=2, $d[3]$=3, $d[4]$=4, $d[5]$=5

- Parents: $p[1]{=}0,\ p[2]{=}1,\ p[3]{=}2,\ p[4]{=}3,\ p[5]{=}4$

Step 2: Build chains (back edges sorted by ancestor depth, shallowest first).

- **Chain 1** from back edge $(2, 0)$: walk $2 \xrightarrow{p} 1 \xrightarrow{p} 0$. Claims tree edges $2{-}1$ and $1{-}0$.
- **Chain 2** from back edge $(4, 2)$: walk $4 \xrightarrow{p} 3 \xrightarrow{p} 2$. Edge $2{-}1$ is already claimed, so chain stops at 2. Claims tree edges $4{-}3$ and $3{-}2$.

Step 3: Check unclaimed tree edges. Tree edge $4{-}5$ is **never claimed** by any chain — no back edge from the subtree rooted at 5 reaches an ancestor of 4.

Conclusion:

- Tree edge $4{-}5$ unclaimed $\Rightarrow$ it is a **bridge** ($\mathrm{BCC}\,3 = \{4, 5\}$).
- Chain 1 covers $\mathrm{BCC}\,1 = \{0, 1, 2\}$.
- Chain 2 covers $\mathrm{BCC}\,2 = \{2, 3, 4\}$.
- Vertices 2 and 4 are **articulation points**.
- **Graph is NOT biconnected** (unclaimed tree edge exists).

2.1.3 Complexity

Property	Bound
Time	$O(V + E)$ — strict, deterministic
Memory	$O(V + E)$

2.2 Slota–Madduri Biconnectivity

2.2.1 Key Idea

Instead of decomposing edges, this algorithm directly identifies **articulation points** using a BFS spanning tree. For each non-root vertex v with BFS-tree children, it checks: "Can each child w reach an ancestor of v without going through v?" This check is done via an **exclusion-BFS** from w in the graph $G \setminus \{v\}$. If the BFS reaches a vertex at level $\leq L[v]$ (that is not v), then w can escape — v is not an AP for that child.

2.2.2 How It Works — Example

Using the **same graph** as above (edges: $0{-}1,\ 0{-}2,\ 1{-}2,\ 2{-}3,\ 2{-}4,\ 3{-}4,\ 4{-}5$):
Step 1: BFS from root 0 produces:

- Level 0: $\{0\}$
- Level 1: neighbours of $0 \Rightarrow \{1, 2\}$
- Level 2: from 1: $\{2\}$ already visited; from 2: $\{3, 4\}$ new
- Level 3: from 3: $\{4\}$ already visited; from 4: $\{5\}$ new
- $L[0]{=}0,\ L[1]{=}L[2]{=}1,\ L[3]{=}L[4]{=}2,\ L[5]{=}3$

- BFS-tree parents: $P[1]=0,\ P[2]=0,\ P[3]=2,\ P[4]=2,\ P[5]=4$
- BFS-tree children: 0: $\{1,2\}$; 2: $\{3,4\}$; 4: $\{5\}$

Step 2: Check each vertex.

- **Root 0** has 2 children $\{1,2\}$. BFS from child 1 (excluding 0): $1 \to 2$ (via edge $1-2$). Child 2 is reached $\Rightarrow$ all children reachable $\Rightarrow$ root is **not AP**.
- **Vertex 2** ($L[2]=1$, children $\{3,4\}$):
  - Exclusion-BFS from 3 (excluding 2): Visit 4 (edge $3-4$). $L[4]=2>1$; visit 5 (edge $4-5$). $L[5]=3>1$. BFS exhausted. No vertex with $L \leq 1$ reached $\Rightarrow$ child 3 **cannot escape**.
  - $\Rightarrow$ **Vertex 2 is an AP**. (Stop checking other children.)
- **Vertex 4** ($L[4]=2$, children $\{5\}$):
  - Exclusion-BFS from 5 (excluding 4): Neighbours of 5: only 4 (excluded). BFS terminates immediately. No vertex with $L \leq 2$ reached $\Rightarrow$ child 5 **cannot escape**.
  - $\Rightarrow$ **Vertex 4 is an AP** (and edge $4-5$ is a bridge).
- **Vertices 1, 3, 5**: no BFS-tree children $\Rightarrow$ skip.
- **Result: 2 articulation points** $\{2,4\}$ — **graph is NOT biconnected**. BCCs: $\{0,1,2\}$, $\{2,3,4\}$, $\{4,5\}$. ✓

2.2.3 Parallelisation

Our implementation uses **OpenMP** to parallelise the exclusion-BFS checks across 16 threads. Each thread maintains its own stamp array, so BFS calls are independent and can run concurrently.

2.2.4 Complexity

Property	Bound
Time (worst case)	$O(V \cdot (V+E))$ — one BFS per child
Time (best case)	$O(V+E)$ — 0 APs, BFS terminates immediately
Memory	$O(V+E)$

3 Implementation Details

3.1 Jens-Schmidt (`Jen-Schmidt-dataset.cpp`)

1. **Iterative DFS** using an explicit stack (avoids recursion stack overflow). Records `parent[]`, `depth[]`, and collects back edges (u,v) where v is an ancestor of u.
2. **Chain building**: back edges sorted by ancestor depth (shallowest first). For each back edge (u,v), walk `parent[]` from u upward, claiming unclaimed tree edges via `chain_id[]`. The chain ends when it reaches v or hits an already-claimed edge.

3. **Biconnectivity decision**:
   - **Biconnected** if every tree edge is claimed by some chain (i.e. number of chains = number of back edges and no unclaimed tree edge remains).
   - **Not biconnected** if any tree edge remains unclaimed after processing all back edges. This means there exists a bridge or articulation point.
4. **Memory tracked**: adjacency list, parent/depth arrays, back-edge vector, chain vectors.

3.2 Slota–Madduri (`slota-dataset.cpp`)

1. **Cycle-graph shortcut**: if every vertex has degree 2 and the graph is connected, it is a cycle $\Rightarrow$ 0 APs, biconnected.
2. **BFS spanning tree**: builds parent $P[]$, level $L[]$ from each connected component's root.
3. **Root AP check**: BFS from the first child of the root (excluding root). If the BFS cannot reach all other children $\Rightarrow$ root is an AP.
4. **Non-root AP check** (OpenMP, 16 threads): for each non-root vertex v with BFS-tree children $\{w_1, \ldots, w_k\}$:
   - Run `exclusion_bfs` from each child w_i in $G \setminus \{v\}$.
   - If BFS reaches a vertex u with $L[u] \leq L[v]$ and $u \neq v$, child w_i **escapes** (can reach an ancestor without v).
   - If **any** child cannot escape $\Rightarrow$ v is an AP. If **all** children escape $\Rightarrow$ v is not an AP.
5. **Biconnectivity decision**:
   - **Biconnected** if number of articulation points = 0.
   - **Not biconnected** if any AP exists.
6. **Memory tracked**: adjacency list, P, L, stamp, seen, isArt arrays.

4 Experimental Setup

4.1 Dataset

81 graphs across 7 categories, all 0-indexed, undirected, no self-loops:

Category	Files	V range	Expected behaviour
small	10	50–500	Small random graphs
sparse	10	1000–10000	$m \approx 1.5n$; many APs; Slota penalised
dense	10	100–1000	$m \approx n^2/2$; few/0 APs; Slota favoured
tree_like	10	1000–10000	$m \approx n$; nearly all internal nodes are APs
highly_connected	10	100–1000	Near-complete graphs; few APs
large	10	1000–10000	Moderate density; performance at scale
real_world	21	1912–54573	Social/collaboration networks

4.2 Methodology

- **Timing**: `std::chrono::high_resolution_clock`, includes I/O. Single run per file.
- **Memory**: analytical sum of `sizeof` + heap capacity.
- **Platform**: Slota uses OpenMP with 16 threads; JS is sequential.

5 Results

5.1 Dense Graphs ($m \approx n^2/2$, $V = 100$–1000)

File	V	E	JS (ms)	Slota (ms)	JS Mem	Sl Mem	APs
dense_01	100	459	0.62	1.54	14536	10432	18
dense_02	200	1909	0.83	2.14	50056	33424	18
dense_03	300	4359	2.16	3.44	85692	50016	18
dense_04	400	7809	3.89	4.63	180780	117808	18
dense_05	500	12259	4.64	5.52	245608	147200	18
dense_06	600	17709	5.52	7.85	319080	176592	18
dense_07	700	24159	9.68	8.77	578660	385184	18
dense_08	800	31609	9.88	10.39	693088	440176	18
dense_09	900	40059	15.58	12.49	816496	495232	18
dense_10	1000	49509	17.93	16.54	945720	550224	18

At $V \geq 700$, **Slota is faster**. With only 18 APs, exclusion-BFS terminates quickly and parallelism pays off. JS uses ∼72% more memory due to massive back-edge storage.

5.2 Highly Connected Graphs ($V = 100$–1000)

File	V	E	JS (ms)	Slota (ms)	JS Mem	Sl Mem	APs
hc_01	100	2451	0.86	1.75	49144	29632	2
hc_02	200	9901	2.93	3.44	189648	110224	2
hc_03	300	22351	7.43	8.00	497368	318816	2
hc_04	400	39801	13.29	14.05	742800	425008	2
hc_05	500	62251	20.16	18.18	1028512	531200	2
hc_06	600	89701	28.83	26.07	1968876	1251792	2
hc_07	700	122151	39.13	32.37	2437352	1460384	2
hc_08	800	159601	50.07	42.49	2944904	1668976	2
hc_09	900	202051	66.22	54.60	3494108	1877632	2
hc_10	1000	249501	75.66	71.70	4082052	2086224	2

Slota wins at $V \geq 500$. Only 2 APs throughout; parallelism dominates. JS uses ∼96% more memory due to ∼248K back edges.

5.3 Large Graphs ($V = 1000$–10000)

File	V	E	JS (ms)	Slota (ms)	JS Mem	Sl Mem	APs
large_01	1000	5001	2.53	3.08	133596	92648	2
large_02	2000	10001	4.36	4.79	268008	185828	3
large_03	3000	15001	7.18	10.84	401304	277520	2
large_04	4000	20001	8.72	9.76	537164	371532	3
large_05	5000	25001	11.59	12.27	670248	464452	3
large_06	6000	30001	14.33	12.85	806224	557648	2
large_07	7000	35001	15.84	17.47	939860	649044	3
large_08	8000	40001	17.80	17.46	1072596	742080	2
large_09	9000	45001	19.36	19.95	1205044	833788	5
large_10	10000	50001	22.38	23.48	1342832	928992	4

Performance is very close (within 5%). With only 2–5 APs, Slota is barely penalised. Slota uses ∼31% less memory consistently.

5.4 Real-World Graphs

Dataset	V	E	JS (s)	Slota (s)	APs
twitch_pt	1912	31299	0.028	0.028	91
fb_tvshows	3892	17239	0.023	0.013	552
facebook	4039	88234	0.047	0.030	11
twitch_ru	4385	37304	0.032	0.017	318
twitch_es	4648	59382	0.040	0.040	239
us_power_grid	4941	6594	0.023	0.005	1228
fb_politician	5908	41706	0.057	0.019	448
eredos	6100	7514	0.032	0.306	447
twitch_fr	6549	112666	0.059	0.048	219
fb_government	7057	89429	0.059	0.042	312
twitch_en	7126	35324	0.049	0.017	780
twitch_de	9498	153138	0.084	0.065	356
fb_public_figure	11565	67038	0.051	0.045	1541
fb_athletes	13866	86811	0.053	0.042	881
fb_company	14113	52126	0.036	0.027	2007
fb_newsites	27917	205964	0.135	0.160	1935
git	37700	289003	0.166	0.123	3187
Deezer_ro	41773	125826	0.078	0.078	5799
Deezer_hu	47538	222887	0.126	0.112	2720
fb_artist	50515	819090	0.409	0.604	2529
Deezer_hr	54573	498202	0.261	0.245	2319

Key observations:

- **eredos** (447 APs, very sparse): JS is 9.5× **faster** — sparse graph + many APs = worst case for Slota.
- **facebook** (11 APs, dense): Slota wins (0.030 vs 0.047 s).
- **git** (3187 APs): Slota faster (0.123 vs 0.166 s) — parallelism compensates for many APs on large graphs.
- **fb_artist** (819K edges): JS 1.5× faster — massive edge count dominates Slota's exclusion-BFS cost.
- **us_power_grid** (1228 APs): Slota 4.6× faster (0.005 vs 0.023 s).

5.5 Small Graphs ($V = 50$–500)

File	V	E	JS (ms)	Slota (ms)	JS Mem	Sl Mem	APs
small_01	50	125	0.37	1.60	4508	3364	1
small_02	100	250	0.42	1.53	9016	6696	2
small_03	150	375	0.30	2.47	13116	9852	7
small_04	200	500	0.39	1.69	18044	13232	6
small_05	250	625	0.45	1.73	22004	16024	4
small_06	300	750	0.49	1.62	26468	19440	2
small_07	350	875	0.55	2.04	31040	22492	3
small_08	400	1000	0.58	1.87	35168	25876	7
small_09	450	1125	0.66	1.98	39968	29256	10
small_10	500	1250	0.63	1.94	44128	32076	7

JS is 3–4× faster. Slota's ∼1.5 ms OpenMP thread startup overhead dominates at this scale.

5.6 Sparse Graphs ($m \approx 1.5n$, $V = 1000$–10000)

File	V	E	JS (ms)	Slota (ms)	JS Mem	Sl Mem	APs
sparse_01	1000	1500	0.71	2.17	67152	52872	189
sparse_02	2000	3000	1.55	2.93	133796	105324	381
sparse_03	3000	4500	1.98	3.68	202108	158016	544
sparse_04	4000	6000	4.87	9.49	267556	210844	764
sparse_05	5000	7500	8.12	8.19	333940	263004	945
sparse_06	6000	9000	7.28	5.90	400988	315208	1140
sparse_07	7000	10500	4.55	6.69	468824	368256	1292
sparse_08	8000	12000	6.17	8.21	534752	420484	1457
sparse_09	9000	13500	7.53	8.78	602344	473436	1637
sparse_10	10000	15000	8.35	10.38	669608	525544	1840

JS is 1.2−3× faster overall. Many APs (∼18% of vertices) force Slota into extensive exclusion-BFS work. Slota uses ∼21% less memory.

5.7 Tree-Like Graphs ($m \approx n$, $V = 1000$–10000)

File	V	E	JS (ms)	Slota (ms)	JS Mem	Sl Mem	APs
tree_01	1000	999	0.88	2.87	49228	47340	504
tree_02	2000	1999	0.96	2.89	98348	94444	993
tree_03	3000	2999	1.32	3.14	147504	141584	1511
tree_04	4000	3999	2.23	4.00	196656	188720	2025
tree_05	5000	4999	2.01	3.51	245668	235780	2521
tree_06	6000	5999	2.29	4.61	295068	283164	3016
tree_07	7000	6999	3.39	5.23	344224	330304	3518
tree_08	8000	7999	2.77	5.25	392756	376820	4007
tree_09	9000	8999	3.43	6.03	442364	424476	4539
tree_10	10000	9999	3.95	6.67	490912	471008	5081

Worst case for Slota: ∼50% of vertices are APs. JS is 1.7−3.3× faster. JS has 0 chains and 0 back edges (pure trees), so memory is nearly identical.

6 Analysis: Expected vs. Observed Behaviour

6.1 Time

Scenario	Expected	Observed
JS on any graph	$O(V + E)$ linear	Confirmed: linear scaling across all categories
Slota, few APs	$O(V + E)$	Confirmed: wins on dense, highly-connected
Slota, many APs	Degrades	Up to 9.5× slower (eredos)
Slota parallel	Speeds up large	Wins on facebook, git, hc, dense
Slota overhead	OMP startup cost	∼1.5 ms baseline on small graphs

Why JS wins on sparse/tree graphs: JS always does $O(V + E)$ work — one DFS pass + one linear chain-building pass. Slota must run exclusion-BFS for every child of every vertex. On trees (∼50% APs), Slota is 1.7−3.3× slower.

Why Slota wins on dense/highly-connected graphs: With few APs (≤ 18), exclusion-BFS terminates quickly. Combined with 16-thread parallelism, Slota outperforms sequential JS on graphs with $V \geq 500$ and $m \gg n$.

6.2 Memory

Both are $O(V + E)$. JS stores more:

Jens-Schmidt	Slota–Madduri
Adjacency list ($2m$ ints)	Adjacency list ($2m$ ints)
parent[], depth[] ($2n$ ints)	P[], L[], stamp[], seen[] ($4n$ ints)
Back edges (up to $m-n+1$ pairs)	isArt[] (n bools)
Chain vectors	—

Slota uses 20–50% less memory because it has no back-edge or chain storage. On highly-connected graphs, JS uses nearly 2× more memory.

7 Plots

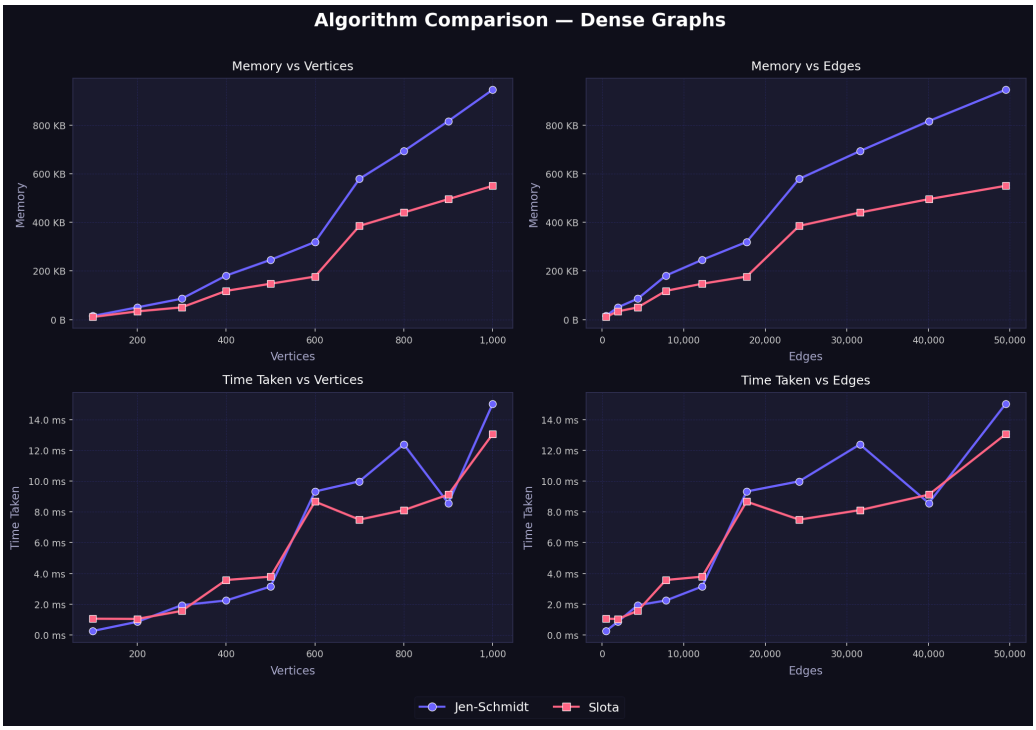


Figure 1: Dense graphs ($m \approx n^2/2$): Slota wins at $V \geq 700$; JS uses ∼72% more memory.

8 Conclusion

Criterion	Jens-Schmidt	Slota–Madduri
Time guarantee	$O(V+E)$ always	$O(V(V+E))$ worst case
Memory	Higher (up to 2×)	Lower (20–50%)
Sparse/tree graphs	1.2–3.3× faster	Penalised by many APs
Dense/HC graphs	Slower at large V	Faster (parallel, few APs)
Large real-world	Wins on sparse nets	Wins on dense social nets
Output	Chain decomposition	Articulation point set
Parallelism	Sequential	OpenMP (16 threads)

Jens-Schmidt is the robust choice: strict $O(V+E)$ regardless of topology, and produces a structural decomposition.

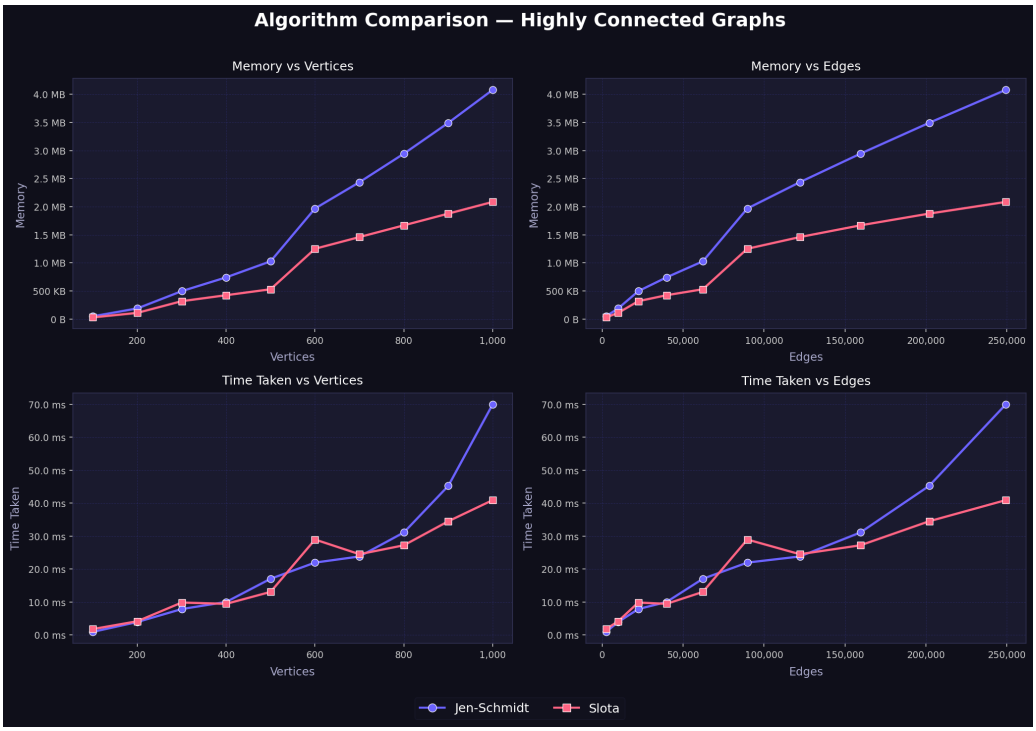


Figure 2: Highly connected (near-K_n): Slota wins at $V \geq 500$; JS uses $\sim$96% more memory.

Slota–Madduri is suitable when memory matters and the graph is known to be dense with few APs, or when parallelism can be exploited on large graphs. On sparse/tree-like inputs, it can be orders of magnitude slower.

References

[1] J. M. Schmidt, *A simple test on 2-vertex- and 2-edge-connectivity*, Information Processing Letters, 113(7):241–244, 2013.

[2] G. Slota and K. Madduri, *Simple Parallel Biconnectivity Algorithms for Multicore Platforms*, IEEE HiPC, 2014.

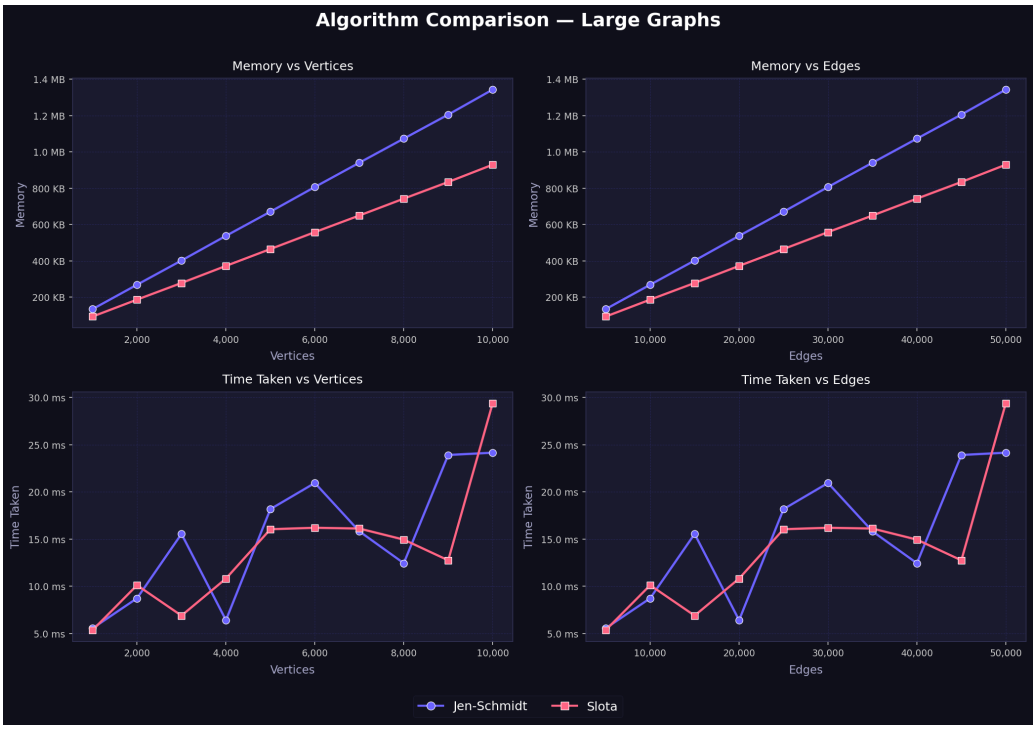


Figure 3: Large graphs (V=1000–10000): nearly identical times; Slota uses ∼31% less memory.

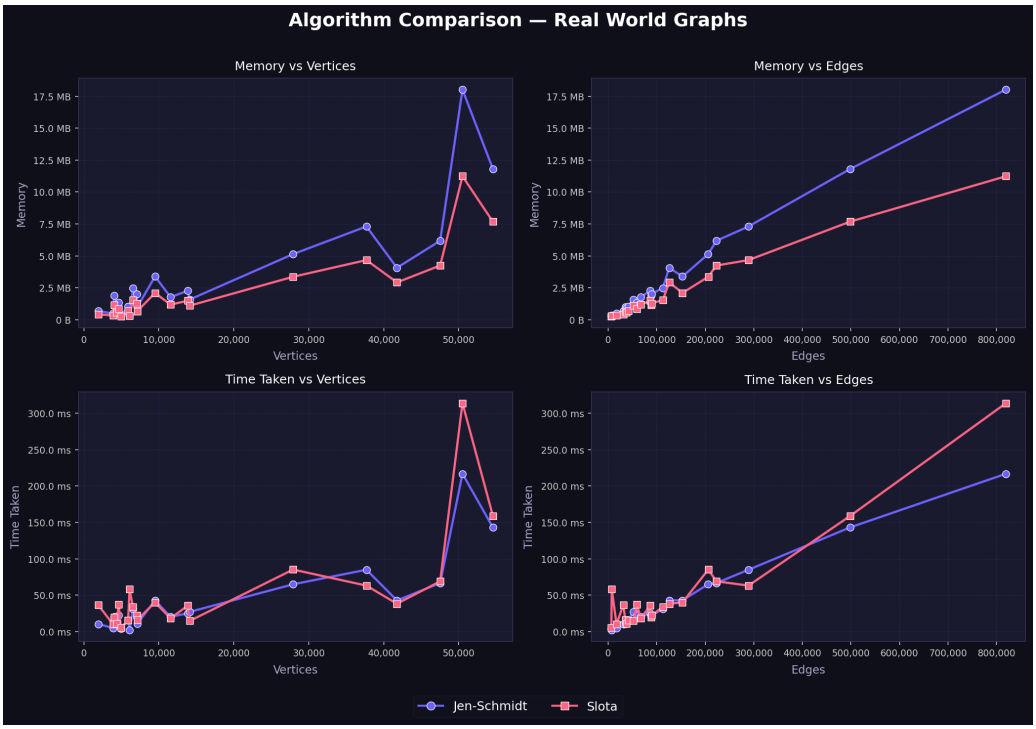


Figure 4: Real-world graphs: Slota wins on facebook, git, twitch; JS wins on eredos, fb_artist.

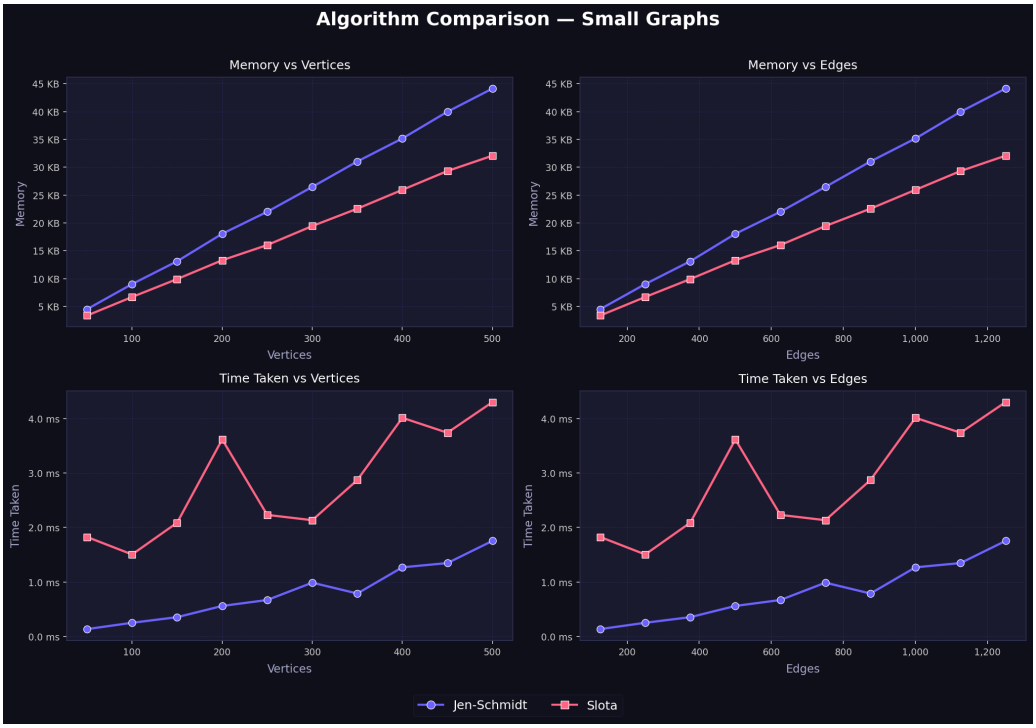


Figure 5: Small graphs: Slota's ∼1.5 ms OpenMP overhead dominates.

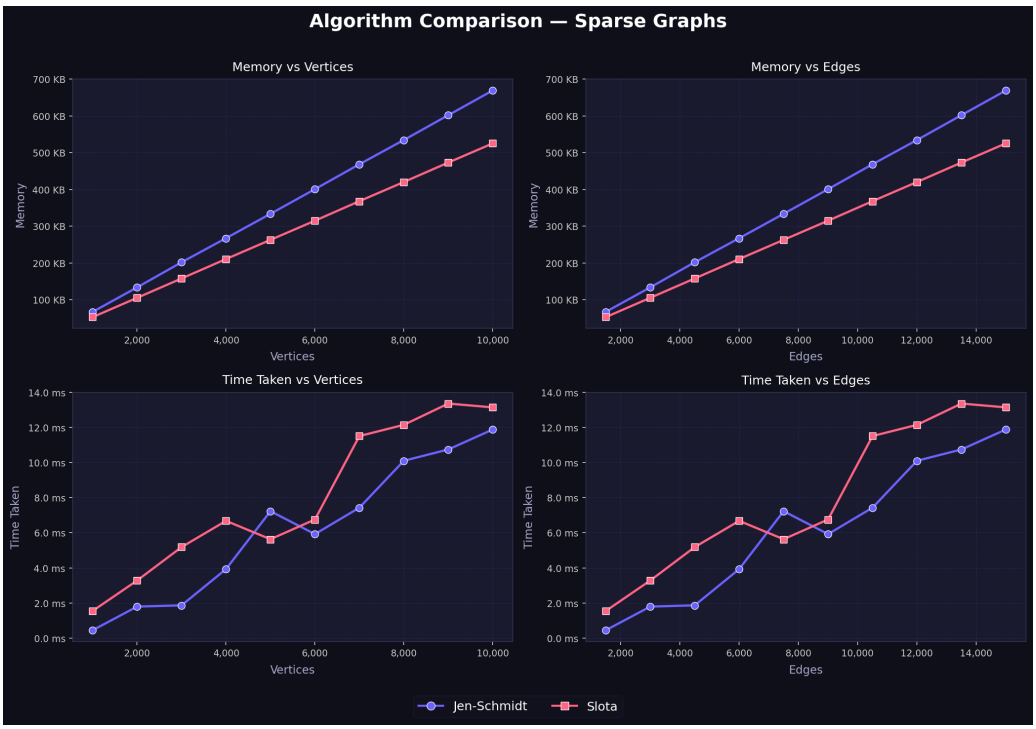


Figure 6: Sparse graphs ($m \approx 1.5n$): JS 1.2–3× faster; ∼18% of vertices are APs.

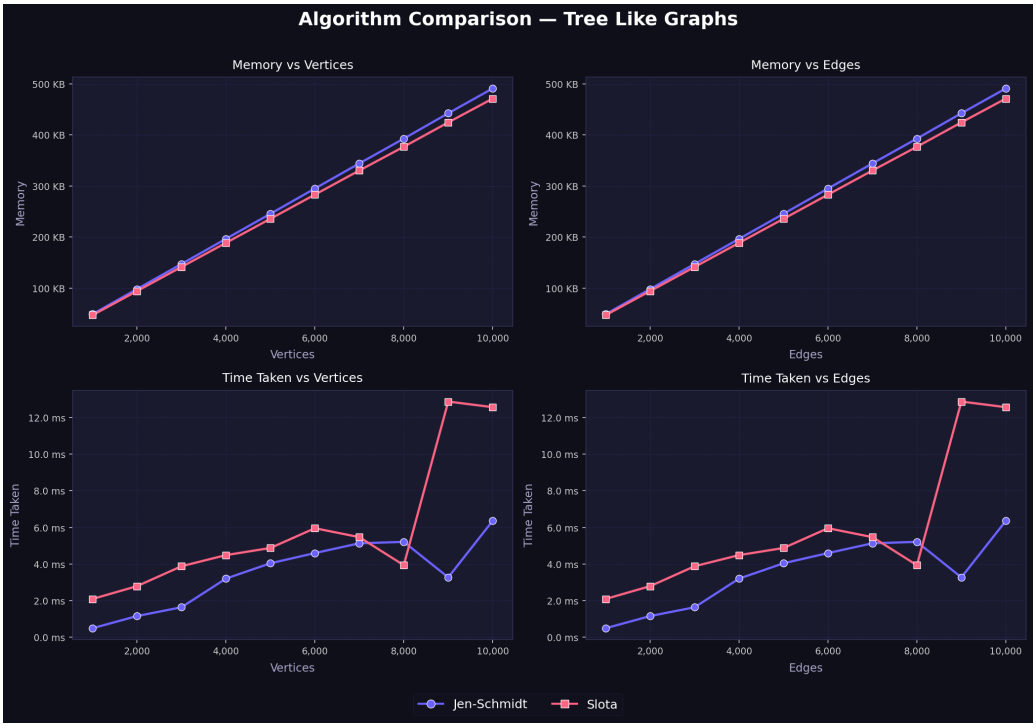


Figure 7: Tree-like graphs ($m \approx n$): JS 1.7–3.3× faster; ∼50% of nodes are APs.